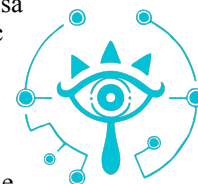


TP 3

Problème

Link voyage à travers le royaume d'Hyrule à la recherche d'anciens artefacts Sheikah. Il commence sa recherche avec un sac magique pouvant contenir jusqu'à 4 artefacts différents. Avec le temps, le sac augmentera automatiquement en capacité afin de stocker encore plus d'articles. Plus précisément, dès qu'au moins $\frac{2}{3}$ des emplacements du sac sont occupés, la capacité du sac est doublée. Il



convient de noter que les éléments sont stockés de manière consécutive, ce qui signifie qu'aucun emplacement "vide" n'est trouvé entre les artefacts stockés. Considérons dans un premier temps que lorsqu'un nouvel artefact est ajouté au sac, il est simplement stocké dans le premier emplacement vide. Pour plus de simplicité, Link a des informations sur la capacité actuelle du sac et l'occupation réelle.

Chaque artefact est décrit par son nom, son pouvoir magique (un entier compris entre 1 et 100), son poids (un entier), la classe de rareté et son prix en rupee (un entier également), et enfin le nombre d'exemplaires (par exemple, deux mêmes orbes magiques occupent une seule case dans le sac).

La classe de rareté va définir un facteur (un réel compris entre 0 et 10) pour l'augmentation de prix lorsque l'objet est vendu à des marchands d'artefacts. Jusqu'à présent, il existe trois classes de rareté d'objet : « non spécifiée » (avec un facteur égal à 0,5, c'est-à-dire que l'artefact est vendu à moitié prix), « standard » (le facteur est de 1,0) et « rare » (le facteur est de 2,5). Il peut y en avoir plus, au fur et à mesure que Link avance dans son aventure. De plus, le facteur d'une classe de rareté peut être modifié avec le temps, et il sera utile qu'il soit automatiquement modifié pour tous les artefacts de cette classe.

Avec l'expérience, Link s'est rendu compte qu'il est pratique de stocker les artefacts par ordre décroissant de puissance magique. Alors, chaque fois à l'ajout d'artefact il maintient l'ordre dans le sac.

Voici un exemple de sac avec capacité 4 et occupation 2 :

Emplacement 1 : Orbe magique, 40 magie, 2 kg, type "non spécifié", 12 rupee par pièce, 4 pièces ;

Emplacement 2 : Arc chasseur, 25 magie, 13 kg, type "rare", 83 rupee par pièce, 1 pièce ;

Emplacement 3 : vide

Emplacement 4 : vide

À la fin du TP, Link doit être capable d'effectuer les opérations suivantes avec son sac :

1. ajouter un artefact ;
2. chercher dans le sac un artefact à partir de son nom et obtenir sa quantité, possiblement 0 ;
3. demander le prix total de tous les artefacts dans le sac ; il s'agit du prix total de vente selon les classes de rareté.
4. trouver l'élément avec le pouvoir magique le plus élevé ;
5. afficher les informations sur tous les artefacts dans le sac ;
6. retirer un artefact du sac ;

Structures de données

On vous propose les structures de données suivantes, mais vous pouvez les modifier ou utiliser d'autres structures de données en les justifiant.

```
Type t_cls_rareté = enregistrement
    c.d.c.    nom
    réel      facteur
Fin enregistrement

Type t_artefact = enregistrement
    c.d.c.    nom
    entier    magique, poids, prix, quantité
    pointeur vers t_cls_rareté classe
Fin enregistrement

Type t_sac = enregistrement
    entier    capacité, occupation
    pointeur vers tab de t_artefact  tab
Fin enregistrement
```

Les tâches à faire

Attention: testez votre code après l'implémentation de chaque fonction.

1. Implémenter les structures de données proposées. Les modifications sont acceptées mais doivent être justifiées.
2. Écrire une fonction d'initialisation de nouveau sac qui crée un nouveau sac de capacité 4 avec le prototype suivant :
`t_sac initSac ()`
3. Écrire deux procédures de redimensionnement du sac avec les prototypes suivants :
`void augmenterCapaSac(t_sac* sac)`
`void reduireCapaSac(t_sac* sac)`
qui vont doubler (ou diviser par deux) la capacité du sac, dont l'adresse est passée en paramètre, si l'occupation atteint 2/3 (ou 1/6 pour la réduction) de sa capacité totale. Attention à la capacité minimale.
4. Écrire une procédure d'affichage d'information d'un artefact sous le format proposé (voir p.1) avec le prototype suivant :
`void afficherArtefact(t_artefact art)`
5. Écrire une procédure d'affichage d'information d'un sac sous le format proposé (voir p.1) avec le prototype suivant :
`void afficherSac(t_sac sac)`
6. Écrire une procédure d'ajout d'un nouvel artefact dans un sac. On suppose le sac trié, et il doit le rester après l'ajout du nouvel élément. Pour rappel, une fois que l'occupation de sac atteint 2/3 de sa capacité, le sac double en taille. La procédure doit obligatoirement avoir le prototype suivant :
`void ajouterArtefact (t_sac* sac, t_artefact art)`
7. Écrire une procédure de suppression d'un artefact, dont le nom est passé en paramètre. Attention à la quantité des artefacts dans chaque emplacement du sac. Réduire la taille du sac une fois l'occupation de sac réduite à 1/6 de sa capacité. Utiliser le prototype suivant :
`bool supprimerArtefact(t_sac* sac, string art_name)`
8. Écrire une fonction de calcul de prix total de tous les artefacts, il s'agit du prix total de vente selon les classes de rareté. Arrondir à l'entier inférieur si nécessaire. La procédure doit obligatoirement avoir le prototype suivant :
`int valeurTotaleVente(t_sac sac)`
9. Écrire une fonction de recherche de l'élément dont le pouvoir magique est le plus élevé, avec le prototype suivant :
`t_artefact maxForceMagique(t_sac sac)`
Proposer un prototype et implémenter une fonction qui retournera un tableau contenant tous les artefacts en cas d'égalité du pouvoir max.
10. Implémenter un scénario de test dans une fonction principale.